

CS3500 LAB3 SUBMISSION

Q1)

Files modified:

kernel/vm.c : wrote the function **vmprint(pagetable_t pt)**

kernel/exec.c : invoked **vmprint** only when `pid == 1`

kernel/defs.h : added prototype for **vmprint**

logic:

Since **xv6** uses a 3-level page table, I implemented **vmprint** recursively to drill down into each level. Each call adds indentation, so the output shows hierarchy clearly. Before printing, I check `PTE_V` to make sure the entry is valid; invalid or unmapped PTEs are skipped. This way the output only shows the useful mappings of a process.

Q2)

Files modified:

kernel/memlayout.h : introduced struct **usyscall** { int pid; } inside the non-assembler guard.

kernel/proc.h : added a pointer to struct **usyscall** inside struct **proc**.

kernel/proc.c :

- In **allocproc()**, allocated a user page at **USYSCALL** with read-only permissions.
- When the process exits, freed the page.
- On fork, wrote the child's PID into this page.
- **user/ulib.c**:

provided **ugetpid()**, which just dereferences the page to fetch PID.

logic:

The design adds a per-process read-only mapping at a fixed address (**USYSCALL**). The kernel initializes this page with the process's PID once. User space does not make a syscall; instead, **ugetpid()** simply reads directly from memory. During cleanup, the mapping is removed so no memory leaks remain.

Q3)

Files modified:

user/user.h : declared **pgaccess()**.

user/usys.pl : added the system call entry.

kernel/riscv.h : defined the access flag PTE_A.

kernel/sysproc.c : implemented sys_pgaccess.

kernel/vm.c : walked the page table to check PTEs and assemble an access bitmask.

kernel/syscall.h : added syscall number for pgaccess.

kernel/syscall.c : linked syscall number to handler.

user/pgtbltest.c : wrote a small test (**pgaccess_test**).

logic:

The new syscall takes a start address, number of pages, and a user buffer. For each page in that range, the kernel calls **walk()** to find its PTE. If the PTE has the PTE_A bit set, the corresponding bit is marked in a kernel bitmask. After recording, the PTE_A flag is cleared so future accesses can be tracked fresh. At the end, the kernel copies this bitmask back to user space via copyout().